

Task 1.1: The Environment State (__init__)

Q1: List the Four Components of the PEAS Framework

Component	Description
Performance Measure (P)	The criterion that defines how successful the agent is, such as an objective function, score, or reward.
Environment (E)	The external world in which the agent operates and interacts.
Actuators (A)	The mechanisms/actions the agent uses to affect or change the environment.
Sensors (S)	The mechanisms the agent uses to perceive information about the environment.

Q2: Which Specific Variables Represent the Physical "Environment (E)" State?

The physical **Environment (E)** state is represented by the following variables:

- `self.width` # Grid width – defines spatial boundaries
- `self.height` # Grid height – defines spatial boundaries
- `self.walls` # Wall coordinates – static obstacles
- `self.food_positions` # Food coordinates – resources in the environment
- `self.opponents` # Opponent positions – other agents in the environment

Environment State Variables

Variable	Description
<code>self.width</code>	Defines the width of the grid.
<code>self.height</code>	Defines the height of the grid.
<code>self.walls</code>	Represents static obstacles within the environment.
<code>self.food_positions</code>	Represents food/resources distributed throughout the environment.
<code>self.opponents</code>	Represents other agents/entities present in the environment.

Variables That Are NOT Environment State

These variables represent the **agent's state or performance**, rather than the physical environment:

Variable	Reason
self.agent_pos	Represents the main agent's own position.
self.score	Represents the agent's performance measure.
self.steps	Keeps track of the number of steps taken by the agent.
self.collision	Represents the agent's collision/status information.

Q3: Classify the Environment as Single-Agent or Multi-Agent

Classification: Multi-Agent Environment

Justification:

1. self.opponents initializes multiple independent agents using num_opponents=2.
2. The opponents have their own positions and exist independently within the environment.
3. The opponents can act as **adversarial agents**, competing with the main agent.
4. The collision flag allows the environment to detect interactions between agents.
5. Multiple agents share and influence the same environment.

Task 1.2: The Perception Subsystem (get_percept)

Q4: Define what a "percept sequence" is

Answer: A percept sequence is the complete history of everything an agent has perceived from the beginning of its operation up to the current time step: [percept₁, percept₂, percept₃, ..., percept_t]

Q5: Which PEAS component does get_percept() represent?

Answer: Sensors

Q6: Fully Observable or Partially Observable?

Answer: Fully Observable

Reason: The agent receives complete information about the entire environment state including its own position, all opponent positions, food detection, wall collisions, collision status, score, remaining food count, and all wall positions from initialization. There is no hidden information.

Task 1.3: Action Execution (execute_action)

Q7: Which PEAS Component Is Being Actively Updated?

Answer: Performance Measure (P)

The agent's performance measure is updated when the agent performs an action, for example, when it hits a wall and receives a penalty.

Q8: Why Is It Structurally Important That This Metric Evaluates Changes in the External Environment State (Hitting a Wall) Rather Than the Agent's Internal Processing Effort?

Answer:

The Performance Measure should evaluate how successfully the agent achieves its goals in the external environment, rather than measuring how much computational effort the agent uses internally.

The agent should be judged based on its actual outcomes and interactions with the environment. For example, hitting a wall can result in a penalty because it represents an unsuccessful action in the environment.

This ensures that the agent is optimized for effective decision-making and interaction with the environment, rather than simply optimizing its internal processing effort.

Q9: How does hiding self.toxic_traps from sensors alter the "Observability" classification?

Answer: It changes the environment from **Fully Observable** to **Partially Observable**.

Reason: The agent no longer has complete information about the environment state because the toxic traps exist in the environment but are hidden from the agent's perception. Since the agent cannot perceive all aspects of the environment, the observability classification becomes partially observable.

Q10: How does 'smells_toxin' help the agent act with "Rationality" (maximizing expected utility)?

Answer: The 'smells_toxin' sensor enables rationality by providing critical information that allows the agent to avoid negative outcomes (toxic traps) and maximize its expected utility through informed decision-making.

Q11: Classic "Vacuum World Exploit" Example

Answer: The classic "Vacuum World Exploit" discussed in Lecture 01 is where a vacuum agent learns to **deliberately push dirt around** or **repeatedly move over the same spot** to generate positive rewards, rather than actually cleaning the room.

Details:

- If the performance measure only rewards "sucking action" (detecting dirt and activating suction), the agent learns to **vacuum the same spot repeatedly** to rack up rewards
- Or it could **push dirt from one location to another** and continuously re-detect it
- This demonstrates that **what you measure is what you optimize** - a flawed metric leads to undesired behavior

How it relates to the lab:

The severe -15 penalty for toxic traps prevents the agent from exploiting the environment by:

- Stepping on traps repeatedly for some unintended benefit
- Trading off small penalties for larger rewards elsewhere
- Ensuring the agent genuinely avoids toxins rather than gaming the system